

MAIE5209 - Design and Analysis of Algorithms

2026-27 Fall Semester - Homework 1

If you have any questions, you can contact armanh@cse.ust.hk

Problem 1: [25 points]

- (a) [15 pts, 3 each] For each pair of expressions (A, B) below, indicate whether A is O , Ω , or Θ of B . Note that zero, one, or more of these relations may hold for a given pair. **List all applicable relations. No explanation is needed.** If $A = \Theta(B)$ then write $A = \Theta(B)$ which already implies that $A = O(B)$ and $A = \Omega(B)$. If $A = \Theta(B)$ and you write $A = O(B)$ or $A = \Omega(B)$ only, you will only receive partial credits. It often happens that some students will get the directions wrong, so please write out the relation in full, i.e., $A = O(B)$, $A = \Omega(B)$, or $A = \Theta(B)$ and not just $O(B)$, $\Omega(B)$ or $\Theta(B)$.

A	B	Relations between A and B
$n^{\log_2 \log_2 n}$	$(\log_2 n)^{\log_2 n}$	
$2^{\sqrt{2 \log_2 n}}$	$\sqrt{n}$	
$n!$	$n^{n/2}$	
$4^{\sqrt{n}}$	2^n	
$\sum_{i=1}^n \frac{1}{i}$	$n \log n$	

- (b) [10 pts] Consider an array A containing n integers. The following algorithm counts the total number of contiguous subarrays whose elements sum to a non-negative value (≥ 0). What is its time complexity? Write your answer in the tightest big-O notation with respect to n .

Algorithm 1 CountNonNegativeSubarrays(A, n)

```

count  $\leftarrow$  0
for  $i \leftarrow 1$  to  $n$  do
    current_sum  $\leftarrow$  0
    for  $j \leftarrow i$  to  $n$  do
        current_sum  $\leftarrow$  current_sum +  $A[j]$ 
        if current_sum  $\geq 0$  then
            count  $\leftarrow$  count + 1
        end if
    end for
end for
return count
    
```

Question 2: [25 pts]

Derive asymptotic upper bounds for $T(n)$ in the following recurrence:

$$T(1) = 1; \quad T(n) = 2T(n/2) + \sqrt{n} \quad (\text{for } n > 1)$$

(You may assume $n = 2^k$ for some positive integer k)

You must Derive the tightest possible asymptotic upper bound for $T(n)$.
Show all your steps using the substitution method to receive full credit.

Question 3: [20 pts]

Given an array of length n , $A[1..n]$, containing real numbers, and an integer parameter k ($1 \leq k \leq n$). We wish to find the maximum sum among all contiguous subarrays of length **at most** k .

Consider the two algorithms *FindMaxBoundedSubarray1* and *FindMaxBoundedSubarray2*.

Algorithm 2 FindMaxBoundedSubarray1(A, n, k)

```
 $n \leftarrow \text{length}(A)$ 
 $max\_sum \leftarrow -\infty$ 
for  $i \leftarrow 1$  to  $n$  do
     $current\_sum \leftarrow 0$ 
     $limit \leftarrow \min(n, i + k - 1)$ 
    for  $j \leftarrow i$  to  $limit$  do
         $current\_sum \leftarrow current\_sum + A[j]$ 
        if  $current\_sum > max\_sum$  then
             $max\_sum \leftarrow current\_sum$ 
        end if
    end for
end for
return  $max\_sum$ 
```

Algorithm 3 FindMaxBoundedSubarray2(A, n, k)

```
 $n \leftarrow \text{length}(A)$ 
 $max\_sum \leftarrow -\infty$ 
Allocate array  $P[0 \dots n]$ 
 $P[0] \leftarrow 0$ 
for  $m \leftarrow 1$  to  $n$  do
     $P[m] \leftarrow P[m - 1] + A[m]$ 
end for
for  $i \leftarrow 1$  to  $n$  do
    for  $j \leftarrow i$  to  $n$  do
        if  $j - i + 1 \leq k$  then
             $sub\_sum \leftarrow P[j] - P[i - 1]$ 
            if  $sub\_sum > max\_sum$  then
                 $max\_sum \leftarrow sub\_sum$ 
            end if
        end if
    end for
end for
return  $max\_sum$ 
```

- (a) [7 points] What is the time complexity of *FindMaxBoundedSubarray1*? Express your answer in terms of n and k .

- (b) **[7 points]** What is the time complexity of *FindMaxBoundedSubarray2*? Express your answer in terms of n and k .
- (c) **[6 points]** When does *FindMaxBoundedSubarray1* run strictly faster than *FindMaxBoundedSubarray2*? When do they exhibit the same asymptotic runtime?

Question 4: [30 pts]

Consider a recursive divide-and-conquer procedure *PartitionScan* that operates on an array of length n . To locate optimal boundary-crossing segments, the algorithm divides the input into two overlapping subproblems of size $n - 1$ (one excluding the first element, one excluding the last element). In addition to the two recursive calls, the algorithm executes 3 constant-time boundary verification steps to reconcile the shared overlap. When $n = 1$, the procedure executes 3 baseline operations without further recursion.

- (a) **[10 pts]** Explain why the recurrence relation for the total number of operations $T(n)$ executed by *PartitionScan* is $T(n) = 2T(n - 1) + 3$, with base case $T(1) = 3$.
- (b) **[10 pts]** Solve the recurrence relation in part (a) using repeated substitution to derive the closed-form expression for $T(n)$.
- (c) **[10 pts]** Use mathematical induction to prove the correctness of your closed-form expression for all integers $n \geq 1$.